

# Open Notebook Light: Vollständige Implementierungs- und Architektur-Dokumentation

Open Notebook Light ist eine schlanke, performante KI-Research-Engine, die auf unnötige externe Container-Infrastrukturen verzichtet und stattdessen auf ein lokal eingebettetes Persistenz- und Modell-Setup unter Linux setzt.

## Vierstufiger Implementierungsfahrplan & Detail-Spezifikation

### Schritt 1: Projekt-Scaffolding & Virtual Environment Setup

- **Quellen-Referenz:** Gestützt auf das *# Advanced Deep Research Prompt: Architectural Feasibility & Greenfield Migration Strategy Audit* sowie das *Comprehensive File-Level Inventory*.
- **Aktivität:** Der Deep Research Agent generiert das vollständige Skript für den bereinigten Verzeichnisbaum und das Manifest. Du kopierst dieses Skript in ein Google Colab Notebook, lässt es dort ausführen und nutzt die Exportfunktion, um die exakt 12 Kern-Dateien aufgeteilt auf die 6 Ordner (`api/`, `open_notebook/ai/`, `database/`, `domain/`, `graphs/`, `utils/`) lokal zu erzeugen.
- **Umgebung:** In der Agenten-IDE wird die Python-Umgebung (`venv`) unter Linux auf Basis von Python 3.11+ und der schlanken `pyproject.toml` (ohne SurrealDB, Podcasts oder AI-Prompter) aufgesetzt.

### Schritt 2: Server-Infrastruktur & Lifecycle (Ohne Businesslogik)

- **Quellen-Referenz:** Abgeleitet aus dem *# Automated Scaffolding & Code Implementation Guide for Open Notebook Light* und dem *File-Level Audit*.
- **Aktivität:** Bereitstellung eines minimalen technischen Fundaments, damit der Server (FastAPI) stabil hochfährt.
- **Komponenten:**
  1. **SQLite-Session-Manager (`sqlite_client.py`):** Asynchrones SQLAlchemy-Setup mit Write-Ahead Logging (`PRAGMA journal_mode=WAL`) und erzwungenen Fremdschlüsseln (`PRAGMA foreign_keys=ON`).
  2. **ChromaDB-Persistent-Client (`vector_store.py`):** Initialisierung des lokalen Vektorspeichers für die `source_chunks`-Collection.
  3. **FastAPI-App-Entrypoint & Lifespan (`api/main.py`):** Lifespan-Hooks führen beim Start automatisch die Tabellen-Erstellung

(`Base.metadata.create_all`) sowie den ChromaDB-Prüf-Call aus.  
Ergänzt um Steuerungs-Skripte (`start.sh`, `stop.sh`) und den  
`/health`-Endpunkt zur Verifikation.

### Schritt 3: Modell-Schicht & Vektor-Businesslogik

- **Quellen-Referenz:** Gestützt auf den *# Hyper-Lean Local RAG Architecture Blueprint (IBM Granite Stack)* und den *Codebase Reduction Audit*.
- **Aktivität:** Implementierung der Daten- und Vektor-Businesslogik gekoppelt an die lokale Modell-Schicht.
- **Komponenten:**
  1. **Lokale Modell-Fabrik (`open_notebook/ai/models.py`):** Zentralisierter Client für den lokalen OpenAI-kompatiblen Endpunkt (Ollama). Bündelt das Chat-Modell (*IBM Granite 4.0 H-Tiny* mit 8k-Kontext) und das Vektor-Modell (*Granite Embedding 107M* mit 384 Dimensionen).
  2. **Relationale ORM-Modelle (`models_sqlite.py`):** Tabellen für Notebooks, Sources, Notes und Chat-Sessions inklusive der Junction-Tabellen (`notebook_sources`, `notebook_notes`).
  3. **Testbarkeit:** Vollständig verifizierbar über reine Kommandozeilen-Skripte im Terminal (SQLite-Schema-Inspektion und In-Script-Schreib-/Lesetests) ganz ohne IDE-Zwang.

### Schritt 4: IDE-Integration, Uploads & Zero-OCR-Fehlerbehandlung

- **Quellen-Referenz:** Abgeleitet aus dem *# OCR Dependency Audit & Non-OCR Processing Feasibility Analysis* und dem *Implementation Blueprint*.
- **Aktivität:** Scharfschalten des Gesamtsystems in der Agenten-IDE, End-to-End-Tests und robuste Fehlerabsicherung ohne schwere Binär-Frameworks (Tesseract-OCR / Poppler-Utils).
- **Komponenten:**
  1. **MIME-Type-Validierung (`api/routers/sources.py`):** Strikte Allowlist am Upload-Endpunkt. Versucht ein Nutzer eine reine Bilddatei (`image/png`, `image/jpeg`) hochzuladen, lehnt das System den Upload sofort mit HTTP `400 Bad Request` ab.
  2. **Ingestion-Absicherung (`open_notebook/graphs/source.py`):** Sollte ein unsearchbares, gescanntes PDF die API-Prüfung passieren, extrahiert der rein pythonbasierte Stream-Parser (`pypdf`) einen leeren Textstring (`""`). Das System bricht kontrolliert ab und setzt den `processing_status` in SQLite sauber auf `failed_no_digital_text`.
  3. **End-to-End-Validierung:** Ausführen von `pytest`-Routentests (Prüfung, dass entfernte Podcast-/Transformations-Endpunkte fehlerfrei `404 Not Found` liefern) sowie Live-Prüfungen des gesamten RAG-Datenflusses von Chunking bis ChromaDB-Vektorabfrage.